

PCAP NetFlow v5 exporter

projekt do predmetu ISA

Tomáš Bordák, *xborda01*

18. novembra 2024

Obsah

1	Uvedenie do problematiky	3
2	Návrh aplikácie	4
3	Popis implementácie	5
4	Základné informácie o programe	6
4.1	Štruktúra pre vstupné argumenty	6
4.2	Štruktúra pre tok	6
4.3	Štruktúra pre spracovávaný paket	7
4.4	Štruktúra pre NetFlow v5 hlavičku	7
4.5	Štruktúra pre NetFlow v5 záznam toku	8
5	Návod na použitie	9
6	Popis testovania aplikácie a výsledku testov	9
7	Prehľad literatúry	12
8	Zdroje	12

2 Návrh aplikácie

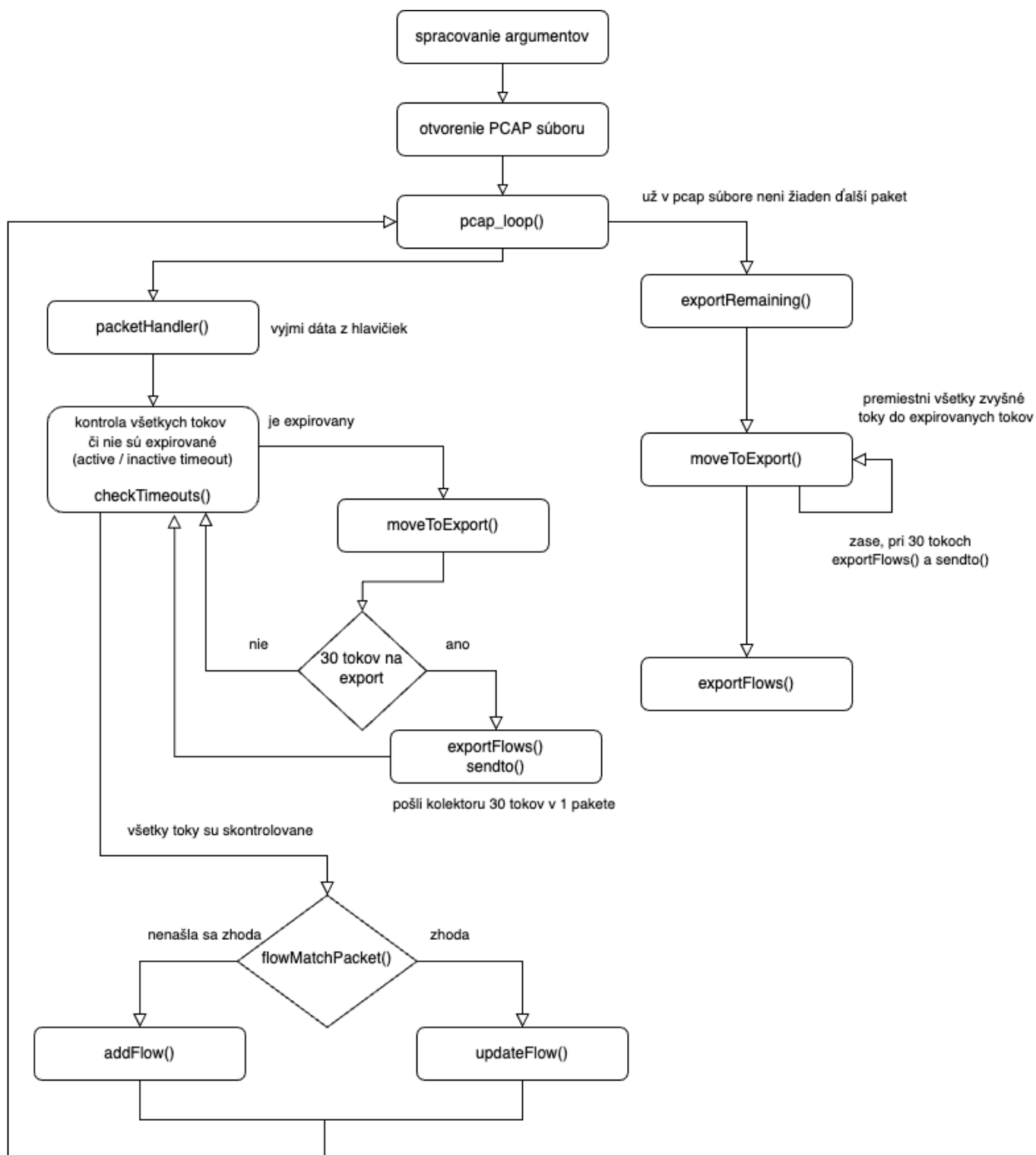
Pri návrhu aplikácie som si najprv musel zistiť informácie o NetFlow, čo to je a ako vlastne funguje. Potom ako som sa dostal do obrazu, začal som zisťovať kroky potrebné pre funkčnú aplikáciu.

Najprv som nevedel kde mám začať a tak som si povedal, že začnem tým najjednoduchším a budem postupne pridávať funkcionality. Takto som postupoval:

- spracovanie argumentov programu
- základná práca s pcap súbormi
- zameranie sa na TCP pakety z pcap suboru
- pridanie funkcií pre pridanie a aktualizácia toku
- pridanie funkcií pre export
- naštudovanie informácií aktívneho a neaktívneho časového limitu toku pre export
- väčšie testovanie a oprava funkcií

Pre verzovanie a ukladanie "checkpointov" som použil nástroj git cez službu GitHub.

3 Popis implementácie



Obr. 2: Implementácia programu

Snažil som sa všetko rozdeliť do rozumne dlhých funkcií, ktoré sú sémanticky pomenované podľa toho čo vykonávajú. Toky sú ukladané do poľa typu `flow_t`, ktorému dynamicky zväčšujem miesto v pamäti, aby mohlo obsiahnuť všetky toky. Expirované toky presúvam do ďalšieho poľa, ktoré má konštantnú veľkosť 30 `flow_t` prvkov, pretože akonáhle pole dosiahne 30 prvkov - teda 30 tokov na export - toky budú poslané na kolektor a pole bude vyprázdnené.

Určite moja implementácia nie je najrýchlejšia a najideálnejšia, ale je funkčná a jednoznačne pochopiteľná.

4 Základné informácie o programe

Dátové modely - štruktúry - používané v tomto programe sú nevyhnutnou súčasťou, už iba napríklad pre ulahčenie práce s vytváraním NetFlow paketou. Štruktúra NetFlow v5 hlavičky a zázname toku, sa dá pekne nadefinovať z formátu, ktorý som našiel na Cisco stránkach.

4.1 Štruktúra pre vstupné argumenty

```
typedef struct
{
    char host[256];
    char *pcapFilePath;
    int port;
    int activeTimeout;
    int inactiveTimeout;
} config_t;
```

4.2 Štruktúra pre tok

```
typedef struct
{
    char srcIP[INET_ADDRSTRLEN];
    char destIP[INET_ADDRSTRLEN];
    uint16_t srcPORT;
    uint16_t destPORT;
    int packetCount;
    int bytesCount;
    struct timeval first;
    struct timeval last;
} flow_t;
```

4.3 Štruktúra pre spracovávaný paket

```
typedef struct
{
char srcIP[INET_ADDRSTRLEN];
char destIP[INET_ADDRSTRLEN];
uint16_t srcPORT;
uint16_t destPORT;
int bytes;
struct timeval timestamp;
} packet_t;
```

4.4 Štruktúra pre NetFlow v5 hlavičku

```
typedef struct
{
uint16_t version;           // version 5

// number of flows exported in this packet (1-30)
uint16_t count;

// current time - boot time in milliseconds
uint32_t SysUptime;

// current time in seconds since 0000 UTC 1970
uint32_t unix_secs;

// current time in nanoseconds since 0000 UTC 1970
uint32_t unix_nsecs;

uint32_t flow_sequence; // sequenceCount
uint8_t engine_type;
uint8_t engine_id;
uint16_t sampling_interval;
} NetFlow_v5_header_t;
```

4.5 Štruktúra pre NetFlow v5 záznam toku

Dátový typ všetkých položiek v štruktúre som zistil z formátu záznamu toku na stránkach spoločnosti Cisco. Všetky podstatné položky pre tento program sú okomentované.

- 1 byte je uint8_t
- 2 bytes je uint16_t
- 4 bytes je uint32_t

```
typedef struct
{
uint32_t srcaddr; // source IP addr
uint32_t dstaddr; // destination IP addr
uint32_t nexthop;
uint16_t input;
uint16_t output;
uint32_t dPkts;    // flow packet count
uint32_t dOctets;  // flow byte count

// first packet time - boot time (in milliseconds)
uint32_t First;

// last packet time - boot time (in milliseconds)
uint32_t Last;

uint16_t srcport; // TCP source port
uint16_t dstport; // TCP destination port
uint8_t pad1;
uint8_t tcp_flags;
uint8_t prot; // TCP = 6
uint8_t tos;
uint16_t src_as;
uint16_t dst_as;
uint8_t src_mask;
uint8_t dst_mask;
uint16_t pad2;
} NetFlow_v5_record_t;
```


Dôležité funkcie z rôznych systémových knižníc, ktoré používam:

- `<pcap/pcap.h >` - `pcap_open_offline()`, `pcap_loop()`
- `<arpa/inet.h >` - `inet_ntop()`, `inet_addr()`, `ntohs()`, `htonl()`
- `<sys/socket.h >` - `sendto()`
- `<sys/time.h >` - `gettimeofday()`

Tým, že NetFlow v5 je protokol, ktorý pracuje s iba IPv4 paketami, som sa rozhodol všetko robiť v IPv4. Program sa nespustí so zadáním IPv6 adresy kolektoru. Podporuje IPv4 adresu kolektoru alebo doménu, ktorá sa preloží na IPv4 adresu (napr. localhost na 127.0.0.1).

5 Návod na použitie

`./p2nprobe host:port pcap_file_path [-a active_timeout -i inactive_timeout -h]`

- `host` (povinný) - IPv4 adresa alebo doména, ktorá sa dá preložiť na IPv4 adresu
- `port` (povinný) - číslo služby, 0 až 65535
- `pcap_file_path` - cesta k pcap súboru
- `-a active_timeout` - počet sekúnd pre nastavenie aktívneho časového limitu. 60 sekúnd, pokiaľ je nenastavený
- `-i inactive_timeout` - počet sekúnd pre nastavenie neaktívneho časového limitu. 60 sekúnd, pokiaľ je nenastavený

6 Popis testovania aplikácie a výsledku testov

Program som testoval priebežne, hlavne pomocou výpiskov na štandardný výstup stdout. Než som implementoval posielanie exportov na kolektor, som si takto overoval dáta, ktoré sa budú posilať.

Ale aby som aj reálne videl, čo sa na kolektor posiela, tak som si v programe Wireshark vytvoril pár pcap súborov. Pár kde boli iba TCP pakety, pre porovnanie oproti programu softflowd (podobný pokročilejší NetFlow exportér) a taktiež pár kde bola všetka komunikácia za danú dobu, aby som otestoval či môj program správne vyberá z paketov, práve tie vrámci TCP protokolu.

Ďalej som používal program `nfcapd` - kolektor, ktorý zbiera NetFlow pakety z exportérov. Pre vizualizáciu prijatých paketov kolektorom som použil program `nfdump`.

Príklady testovania s postupom a výsledkom:

- zapnúť kolektor: **nfcapd -v -l . -p 2055 -b localhost -4** (-4 pre IPv4)
- **./p2nprobe localhost:2055 test_pcap_files/druhy_pcap.pcap**
- vypnúť kolektor pomocou control + C (na macOS)
- zapnúť znova kolektor pre softflowd: **nfcapd -v -l . -p 2055 -b localhost**
- **./softflowd -r ../Desktop/ISA24/test_pcap_files/druhy_pcap.pcap -n localhost:2055 -v 5 -d -P udp**
- vypnúť kolektor pomocou control + C (na macOS)

Vypnutím kolektora sa uložia prijaté NetFlow pakety.

Následne pomocou **nfdump -r nfcapd.202411180735** sa vypíše na štandardný výstup stdout čo bolo kolektorom prijaté, samotné dáta a ďalšie štatistiky. Takto isto som si vypísal uložené súbory kolektorom pre môj program aj softflowd.

```
kalasi@MacBook-Air /test_pcap_files % nfcapd -v -l . -p 2055 -b localhost -4
-l is a legacy option and may get removed in future. Please use -w to set output directory
Verbose log level: 1
Bound to IPv4 host/IP: localhost, Port: 2055
Init v5/v7: Default sampling: 1
Init v9: Max number of v9 tags enabled: 114, default sampling: 1
Init IPFIX: Max number of ipfix tags enabled: 113, default sampling: 1
Startup nfcapd.
Process_v5: New exporter: SysID: 1, engine id 0, type 0, IP: 127.0.0.1
^CIdent: 'none' Flows: 26, Packets: 184, Bytes: 70449, Sequence Errors: 0, Bad Packets: 0, Blocks: 1
Total packets received: 1 avg: 0.00 ignored packets: 0
Terminating nfcapd.
kalasi@MacBook-Air /test_pcap_files % nfdump -r nfcapd.202411182300
Date first seen      Duration      Proto      Src IP Addr:Port      Dst IP Addr:Port      Packets      Bytes      Flows
2024-11-18 05:25:34.989 00:00:30.833 TCP        192.168.31.56:57024 -> 192.168.31.112:8009      14      1498      1
2024-11-18 05:25:34.993 00:00:30.830 TCP        192.168.31.56:57023 -> 192.168.31.112:8009      14      1498      1
2024-11-18 05:25:35.114 00:00:30.708 TCP        192.168.31.112:8009 -> 192.168.31.56:57024      7      1134      1
2024-11-18 05:25:35.114 00:00:30.709 TCP        192.168.31.112:8009 -> 192.168.31.56:57023      7      1134      1
2024-11-18 05:25:36.611 00:00:01.051 TCP        192.168.31.56:59067 -> 152.199.19.160:443      23      4566      1
2024-11-18 05:25:36.627 00:00:01.035 TCP        152.199.19.160:443 -> 192.168.31.56:59067      45      53814      1
2024-11-18 05:25:37.081 00:00:09.693 TCP        192.168.31.56:58758 -> 162.159.130.234:443      6      366      1
2024-11-18 05:25:37.096 00:00:09.678 TCP        162.159.130.234:443 -> 192.168.31.56:58758      6      727      1
2024-11-18 05:25:38.074 00:00:00.030 TCP        192.168.31.56:55868 -> 35.186.224.44:443      2      147      1
2024-11-18 05:25:38.094 00:00:00.010 TCP        35.186.224.44:443 -> 192.168.31.56:55868      2      144      1
2024-11-18 05:25:40.404 00:00:00.000 TCP        192.168.31.56:56776 -> 147.229.176.19:22      1      104      1
2024-11-18 05:25:40.417 00:00:00.000 TCP        147.229.176.19:22 -> 192.168.31.56:56776      1      52      1
2024-11-18 05:25:42.049 00:00:09.642 TCP        192.168.31.56:59013 -> 52.111.243.15:443      3      173      1
2024-11-18 05:25:42.081 00:00:09.692 TCP        52.111.243.15:443 -> 192.168.31.56:59013      3      181      1
2024-11-18 05:25:46.774 00:00:10.374 TCP        54.183.196.5:443 -> 192.168.31.56:55875      4      286      1
2024-11-18 05:25:46.774 00:00:10.374 TCP        192.168.31.56:55875 -> 54.183.196.5:443      4      304      1
2024-11-18 05:25:47.627 00:00:00.030 TCP        192.168.31.56:55871 -> 35.186.224.44:443      10      808      1
2024-11-18 05:25:47.643 00:00:00.014 TCP        35.186.224.44:443 -> 192.168.31.56:55871      5      358      1
2024-11-18 05:25:58.347 00:00:00.009 TCP        157.240.30.32:5222 -> 192.168.31.56:56613      2      143      1
2024-11-18 05:25:58.347 00:00:00.000 TCP        192.168.31.56:56613 -> 157.240.30.32:5222      2      133      1
2024-11-18 05:26:05.011 00:00:00.085 TCP        192.168.31.56:59068 -> 192.168.31.112:8008      6      511      1
2024-11-18 05:26:05.019 00:00:00.077 TCP        192.168.31.112:8008 -> 192.168.31.56:59068      4      1410      1
2024-11-18 05:26:05.089 00:00:00.076 TCP        192.168.31.56:59069 -> 192.168.31.112:8008      6      520      1
2024-11-18 05:26:05.094 00:00:00.071 TCP        192.168.31.112:8008 -> 192.168.31.56:59069      4      260      1
2024-11-18 05:26:06.173 00:00:00.030 TCP        192.168.31.56:55848 -> 34.158.1.133:4070      2      115      1
2024-11-18 05:26:06.203 00:00:00.000 TCP        34.158.1.133:4070 -> 192.168.31.56:55848      1      63      1
Summary: total flows: 26, total bytes: 70449, total packets: 184, avg bps: 18055, avg pps: 5, avg bpp: 382
Time window: 2024-11-18 05:25:34 - 2024-11-18 05:26:06, Duration: 00:00:32.000
Total records processed: 26, passed: 26, Blocks skipped: 0, Bytes read: 2248
Sys: 0.0050s User: 0.0022s Wall: 0.0012s flows/second: 22508.1 Runtime: 0.0012s
kalasi@MacBook-Air /test_pcap_files %
```

Obr. 3: Výstup môjho programu p2nprobe nad druhy_pcap.pcap

```

kalasi@MacBook-Air /test_pcap_files % nfcapd -v -l . -p 2055 -b localhost
-l is a legacy option and may get removed in future. Please use -w to set output directory
Verbose log level: 1
Bound to IPv6 host/IP: localhost, Port: 2055
Init v5/v7: Default sampling: 1
Init v9: Max number of v9 tags enabled: 114, default sampling: 1
Init IPFIX: Max number of ipfix tags enabled: 113, default sampling: 1
Startup nfcapd.
Process_v5: New exporter: SysID: 1, engine id 0, type 0, IP: ::1
^CIdent: 'none' Flows: 26, Packets: 184, Bytes: 70449, Sequence Errors: 0, Bad Packets: 0, Blocks: 1
Total packets received: 1 avg: 0.00 ignored packets: 0
Terminating nfcapd.
kalasi@MacBook-Air /test_pcap_files % nfdump -r nfcapd.202411182305
Date first seen      Duration      Proto      Src IP Addr:Port      Dst IP Addr:Port      Packets      Bytes      Flows
2024-11-18 05:25:36.612 00:00:01.050 TCP        152.199.19.160:443    -> 192.168.31.56:59067      45      53814      1
2024-11-18 05:25:36.612 00:00:01.050 TCP        192.168.31.56:59067    -> 152.199.19.160:443      23      4566      1
2024-11-18 05:26:05.011 00:00:00.085 TCP        192.168.31.56:59068    -> 192.168.31.112:8008      6       511      1
2024-11-18 05:26:05.011 00:00:00.085 TCP        192.168.31.112:8008    -> 192.168.31.56:59068      4      1410      1
2024-11-18 05:26:05.089 00:00:00.076 TCP        192.168.31.56:59069    -> 192.168.31.112:8008      6       520      1
2024-11-18 05:26:05.089 00:00:00.076 TCP        192.168.31.112:8008    -> 192.168.31.56:59069      4       260      1
2024-11-18 05:25:38.075 00:00:00.029 TCP        35.186.224.44:443     -> 192.168.31.56:55868      2       144      1
2024-11-18 05:25:38.075 00:00:00.029 TCP        192.168.31.56:55868    -> 35.186.224.44:443      2       147      1
2024-11-18 05:25:40.404 00:00:00.014 TCP        147.229.176.19:22     -> 192.168.31.56:56776      1        52      1
2024-11-18 05:25:40.404 00:00:00.014 TCP        192.168.31.56:56776    -> 147.229.176.19:22      1       104      1
2024-11-18 05:25:37.081 00:00:09.693 TCP        162.159.130.234:443    -> 192.168.31.56:58758      6       727      1
2024-11-18 05:25:37.081 00:00:09.693 TCP        192.168.31.56:58758    -> 162.159.130.234:443      6       366      1
2024-11-18 05:25:47.627 00:00:00.031 TCP        35.186.224.44:443     -> 192.168.31.56:55871      5       358      1
2024-11-18 05:25:47.627 00:00:00.031 TCP        192.168.31.56:55871    -> 35.186.224.44:443     10       808      1
2024-11-18 05:25:42.049 00:00:09.724 TCP        52.111.243.15:443     -> 192.168.31.56:59013      3       181      1
2024-11-18 05:25:42.049 00:00:09.724 TCP        192.168.31.56:59013    -> 52.111.243.15:443      3       173      1
2024-11-18 05:25:46.774 00:00:10.375 TCP        54.183.196.5:443      -> 192.168.31.56:55875      4       286      1
2024-11-18 05:25:46.774 00:00:10.375 TCP        192.168.31.56:55875    -> 54.183.196.5:443      4       304      1
2024-11-18 05:25:58.347 00:00:00.010 TCP        157.240.30.32:5222    -> 192.168.31.56:56613      2       143      1
2024-11-18 05:25:58.347 00:00:00.010 TCP        192.168.31.56:56613    -> 157.240.30.32:5222      2       133      1
2024-11-18 05:25:34.989 00:00:30.833 TCP        192.168.31.56:57024    -> 192.168.31.112:8009     14      1498      1
2024-11-18 05:25:34.989 00:00:30.833 TCP        192.168.31.112:8009    -> 192.168.31.56:57024      7      1134      1
2024-11-18 05:25:34.994 00:00:30.829 TCP        192.168.31.56:57023    -> 192.168.31.112:8009     14      1498      1
2024-11-18 05:25:34.994 00:00:30.829 TCP        192.168.31.112:8009    -> 192.168.31.56:57023      7      1134      1
2024-11-18 05:26:06.174 00:00:00.030 TCP        34.158.1.133:4070     -> 192.168.31.56:55848      1        63      1
2024-11-18 05:26:06.174 00:00:00.030 TCP        192.168.31.56:55848    -> 34.158.1.133:4070      2       115      1
Summary: total flows: 26, total bytes: 70449, total packets: 184, avg bps: 18055, avg pps: 5, avg bpps: 382
Time window: 2024-11-18 05:25:34 - 2024-11-18 05:26:06, Duration: 00:00:32.000
Total records processed: 26, passed: 26, Blocks skipped: 0, Bytes read: 2560
Sys: 0.0052s User: 0.0023s Wall: 0.0011s flows/second: 23401.7 Runtime: 0.0011s
kalasi@MacBook-Air /test_pcap_files %

```

Obr. 4: Výstup programu softflowd nad druhý_pcap.pcap

Zachytené pakety som skúmal aj v programe Wireshark po nastavení interface na Loopback. Pre skrátenie tejto dokumentácie nebudem uvádzať viac príkladov. Na serveri merlin.fit.vutbr.cz testované spustenie a výpisky.

7 Prehľad literatúry

- NetFlow Export Datagram Format
- prednáška Monitorování toků Netflow - z roku 2023/2024 od slidu 301

8 Zdroje

- Obr.1 - <https://www.plixer.com/blog/netflow-overview-advanced-netflow-traffic-analyzer/>